Clase 6. Arquitectura Orientada a Eventos (EDA) en Microservicios

Objetivo

Comprender, analizar y aplicar los principios, patrones y mecanismos fundamentales de la Arquitectura Orientada a Eventos (EDA) en el contexto de microservicios modernos, evaluando su impacto en la escalabilidad, resiliencia, trazabilidad, consistencia eventual y desacoplamiento de sistemas distribuidos; y relacionar estos conceptos con patrones avanzados como Saga, Event Bus, monitoreo distribuido y mensajería empresarial, para su aplicación en escenarios reales como el proyecto Amazon Distribuido.

Introducción

La Arquitectura Orientada a Eventos (Event-Driven Architecture, EDA) se ha convertido en uno de los paradigmas más determinantes en el diseño de sistemas distribuidos modernos. A diferencia de los modelos centrados en llamadas directas o en flujos secuenciales, la EDA propone que los microservicios se comuniquen mediante la emisión, propagación y consumo de eventos que representan hechos significativos en el dominio. Este enfoque habilita una forma de interacción asincrónica, reactiva y altamente escalable, en la que los microservicios no dependen entre sí, sino que responden a señales del ecosistema. Gracias a ello, las plataformas contemporáneas —como Amazon, Netflix o MercadoLibre— pueden manejar millones de transacciones y eventos por segundo manteniendo estabilidad, resiliencia y evolución continua.

En esta clase se analizan en profundidad los elementos centrales de la EDA: los patrones de coordinación Coreografía y Orquestación, el patrón Saga para consistencia eventual, las plataformas de mensajería y streaming como RabbitMQ y Kafka, los principios de desacoplamiento mediante eventos y los mecanismos avanzados de monitoreo y trazabilidad necesarios para operar un sistema distribuido real. A través de estos componentes, la clase permite comprender no solo cómo se construyen sistemas event-driven, sino también cómo se gobiernan, cómo se detectan fallos, cómo se escalan y cómo convergen hacia estados coherentes en presencia de asincronía. Este marco teórico se aplica directamente al proyecto Amazon Distribuido, donde los estudiantes modelan y experimentan con flujos reales de negocio basados totalmente en eventos.

La Arquitectura Orientada a Eventos (Event-Driven Architecture, EDA) constituye probablemente el paradigma más influyente en la evolución contemporánea de los sistemas distribuidos. A diferencia de los enfoques tradicionales centrados en llamadas directas (RPC, REST) o integraciones secuenciales controladas por un servicio central (SOA clásico), la EDA se sustenta en un principio más natural y más cercano a los sistemas del mundo real: la reacción a eventos.

Un evento es cualquier hecho significativo que altera el estado de un sistema, como *PedidoCreado*, *PagoAprobado*, *UsuarioRegistrado*, *InventarioAgotado*, entre otros. En arquitecturas modernas, estos eventos se convierten en la unidad mínima de comunicación y coordinación entre microservicios.

Orígenes de la EDA

La EDA no es una invención reciente: tiene raíces en los modelos de programación reactiva, la teoría de autómatas y los sistemas de mensajería de los años 90 (MQSeries, JMS). Sin embargo, su consolidación ocurre con la masificación de plataformas como Kafka, RabbitMQ,

ActiveMQ, y con el avance del cómputo distribuido a gran escala en nubes públicas.
Las organizaciones descubrieron que el acoplamiento inherente a las llamadas directas era una limitación para la escalabilidad extrema. Amazon, por ejemplo, migró a un modelo event-driven porque cada call chain REST tradicional se convertía en un cuello de botella durante picos como Black Friday.

El problema central que resuelve la EDA

En un sistema distribuido con decenas de microservicios, cada uno con responsabilidad individual, se presentan múltiples desafíos:

1. ¿Cómo coordinar múltiples servicios sin centralizar el flujo?
2. ¿Cómo garantizar consistencia sin transacciones distribuidas?
3. ¿Cómo evitar que fallas aisladas afecten a todo el sistema?
4. ¿Cómo manejar miles o millones de eventos por segundo?
5. ¿Cómo permitir que cada servicio evolucione independientemente?

La respuesta moderna a estos desafíos es la EDA, la cual logra:

- Desacoplamiento temporal: los productores no dependen del estado del consumidor.
- Desacoplamiento espacial: los servicios no necesitan conocerse entre sí.
- Desacoplamiento semántico: un evento puede significar diferentes cosas para distintos consumidores.
- Alta resiliencia: el sistema sigue funcionando incluso ante fallas parciales.
- Consistencia eventual: los subsistemas convergen a un estado coherente sin transacciones globales.

EDA como columna vertebral del comercio electrónico moderno

Plataformas como Amazon, Alibaba, MercadoLibre, y Walmart construyen sus sistemas sobre EDA porque:

- Los flujos comerciales son masivos, concurrentes y distribuidos.
- La latencia debe mantenerse mínima.
- La tolerancia a fallos es fundamental.
- La personalización requiere procesar millones de eventos de navegación.
- El ML/AI en tiempo real exige streams permanentes de datos.

Componentes esenciales de la EDA

Una arquitectura orientada a eventos incluye:

- **Productores:** generan eventos.
- **Consumidores:** reaccionan a eventos de interés.
- **Event Bus:** infraestructura que transporta eventos (Kafka, RabbitMQ).

- **Event Store:** almacenamiento de eventos históricos (Kafka log).
- **Procesadores:** transforman, filtran o agregan eventos.
- **Orquestadores o Sagas:** coordinan flujos complejos.

6.1. Coreografía vs Orquestación

El proceso de coordinación entre microservicios constituye uno de los pilares más críticos de la Arquitectura Orientada a Eventos (EDA). En arquitecturas distribuidas, los microservicios funcionan como unidades autónomas, cada uno con su base de datos, su lógica interna, sus reglas operativas y su ciclo de despliegue. Esta independencia aporta enormes ventajas en escalabilidad, resiliencia, despliegue continuo y reducción del acoplamiento; sin embargo, introduce un problema fundamental: *¿cómo lograr que microservicios autónomos cooperen en procesos de negocio que abarcan múltiples pasos y múltiples actores?*

Los patrones de coreografía y orquestación emergen como dos respuestas antagónicas pero complementarias. La **coreografía** se inspira en sistemas biológicos y en arquitecturas reactivas: no existe una entidad central que controle todo; las interacciones ocurren mediante señales (eventos) que permiten que el comportamiento global emerja. La **orquestación**, en cambio, responde a un modelo más tradicional: un componente centralizado actúa como maestro de ceremonias, dirigiendo el flujo, invocando microservicios y decidiendo qué ocurre ante fallos.

La diferencia entre ambos no es trivial: determina la naturaleza del sistema, su escalabilidad, su resiliencia, su mantenibilidad y, sobre todo, la forma en la que evoluciona.

1. Coreografía: un patrón distribuido basado en eventos

El patrón de coreografía consiste en una coordinación 100% descentralizada. Ningún servicio tiene la responsabilidad exclusiva de controlar el flujo global. En vez de ello, cada microservicio:

1. Escucha eventos relevantes para su dominio.
2. Ejecuta su lógica local (una transacción ACID).
3. Publica nuevos eventos como consecuencia natural de su operación.

Este modelo es extremadamente potente por tres razones fundamentales:

1.1. Desacoplamiento extremo

En la coreografía, un servicio no necesita saber quién consumirá los eventos que genera. Esto evita dependencias directas, elimina la necesidad de contratos rígidos entre servicios y permite que múltiples consumidores se agreguen o eliminen sin modificar el productor.

Este desacoplamiento no es solo estructural: es temporal.
El productor publica eventos sin preocuparse de si el consumidor está disponible en ese momento. Esta asincronía es clave para soportar picos de carga masivos.

1.2. Escalabilidad elástica

Las arquitecturas basadas en eventos tienden a escalar de forma natural porque:

- Cada microservicio puede replicarse tantas veces como requiera.

- El Event Bus (Kafka, RabbitMQ, Pulsar, AWS Kinesis) gestiona presión y backpressure.
- Los consumidores pueden procesar eventos a su propio ritmo, sin bloquear al productor.

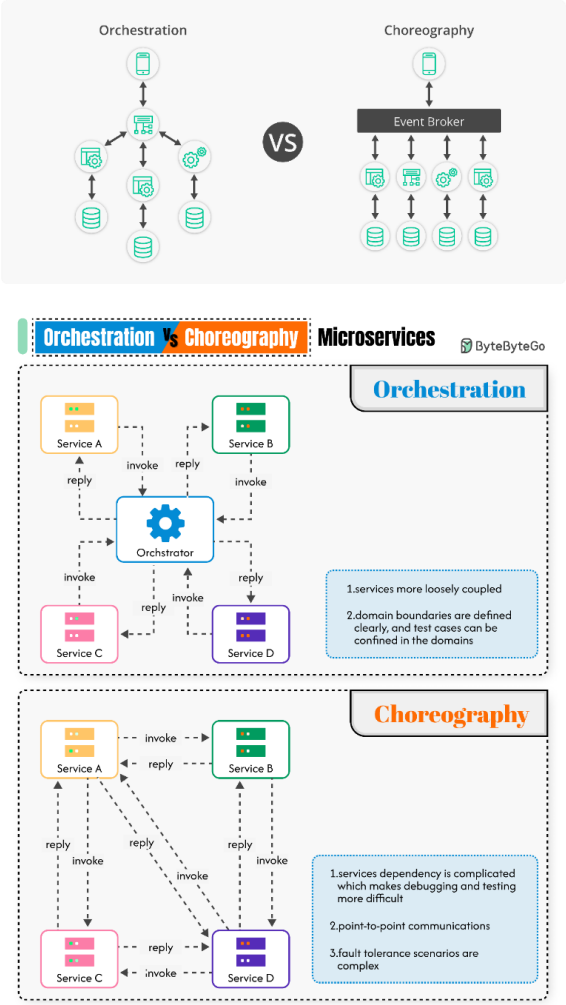
Por esta razón, Amazon utiliza un enfoque coreografiado para manejar inventarios, analíticas, navegación, recomendaciones y telemetría.

1.3. Emergencia del comportamiento global

La característica más notable de la coreografía es que el flujo no está predefinido en un único elemento del sistema. El proceso completo se "descubre" observando cómo los eventos se propagan y cómo los servicios reaccionan.
Esta propiedad se denomina comportamiento emergente y representa una ventaja y un reto:

- Ventaja: el sistema puede crecer modularmente y adaptarse a nuevas necesidades.
- Reto: el debugging requiere herramientas avanzadas de observabilidad.

Figura 1. Coreografía de microservicios basada en eventos.



Fuente: Elaboración propia (2025).

2. Orquestación: control explícito y centralizado del proceso

El patrón de orquestación define una arquitectura donde un componente central (un orquestador) controla el flujo completo del proceso. El orquestador:

- Decide qué microservicio ejecutar.
- Maneja los errores.
- Ejecuta transacciones compensatorias.
- Mantiene la secuencia.

Este enfoque se asemeja más a un tradicional workflow empresarial o un BPMN.

2.1. Claridad y visibilidad del proceso

Una ventaja clave es que el flujo completo del proceso se puede visualizar fácilmente como un diagrama secuencial:

"1) Crear pedido → 2) Reservar inventario → 3) Procesar pago → 4) Notificar".

Esto es fundamental cuando se necesita auditoría rigurosa, como en:

- Pagos electrónicos.
- Procesos contables.
- Procesos financieros regulados.
- Flujos antifraude.
- Procesos fiscales.

2.2. Manejo centralizado de errores

A diferencia de la coreografía, donde los errores se manejan en forma distribuida, la orquestación permite ejecutar:

- Reintentos controlados.
- Timeouts estrictos.
- Compensaciones predefinidas.
- Políticas diagnósticas claras.

Esto proporciona mayor predictibilidad en flujos críticos.

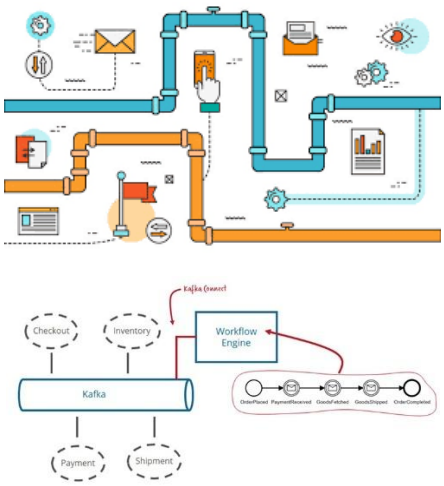
2.3. Riesgos y limitaciones

El patrón de orquestación introduce desafíos importantes:

- Si no se diseña cuidadosamente, el orquestador se convierte en un mini-monolito.
- El sistema es vulnerable si el orquestador falla.
- Cada microservicio depende implícitamente del orquestador.
- Reduce la flexibilidad del ecosistema.

Por esta razón, muchas plataformas usan orquestación solo para procesos regulados y dejan el resto a un modelo coreografiado.

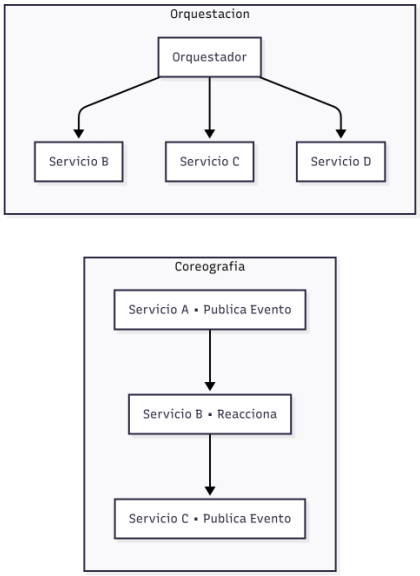
Figura 2. Orquestación de microservicios mediante un controlador de flujo.



Fuente: Creación propia (2025).

3. Comparación operativa

Figura 3. Diagrama operativo comparado entre coreografía y orquestación.



Fuente: Elaboración propia (2025).

4. Análisis arquitectónico

4.1. Modelo de propagación del estado

En la coreografía, el estado global se propaga por eventos.
Esto significa que el sistema adopta un modelo que se asemeja a los sistemas de eventos de la biología (sinapsis, hormonas, señales químicas). Cada servicio actúa de manera autónoma con base en la información disponible.

En la orquestación, el estado se concentra en el orquestador, que “posee el conocimiento” del proceso.

4.2. Impacto en la consistencia

En la coreografía:

- No existe consistencia inmediata.
- La consistencia eventual es la norma.
- Los servicios convergen con el tiempo.

En la orquestación:

- Puede simularse consistencia más estricta mediante control centralizado.

4.3. Complejidad algorítmica

La coreografía distribuye la complejidad entre servicios.
La orquestación concentra la complejidad en el orquestador.

Este contraste cambia completamente el diseño del sistema y su mantenibilidad.

5. Tabla comparativa técnica

Tabla 1. Comparación profunda entre coreografía y orquestación en microservicios.

Dimensión	**Coreografía**	**Orquestación**
Control	Distribuido	Centralizado
Flujo	Emergente	Determinístico
Escalabilidad	Muy alta	Media-alta
Trazabilidad	Compleja	Clara
Fallos	Gestionados por servicios	Gestionados por orquestador
Flexibilidad	Máxima	Media
Regulación	Limitada	Ideal
Evolución	Independiente	Moderada
Necesidad de Event Bus	Obligatoria	Opcional

Fuente: Elaboración propia (2025).

Conclusión

La decisión entre coreografía y orquestación no debe basarse en preferencias subjetivas, sino en el contexto del proceso, la criticidad de los datos, los volúmenes de eventos, las necesidades regulatorias y los objetivos de escalabilidad. Ambos patrones no compiten entre sí: se complementan.
En sistemas como Amazon, pedidos, inventarios y notificaciones siguen un modelo coreografiado; pagos, reembolsos y antifraude se manejan mediante orquestación. Esta

combinación permite construir sistemas robustos, coherentes, escalables y preparados para crecimiento exponencial.

6.2. Patrón Saga: Compensaciones y Consistencia Eventual

El patrón Saga es una de las piezas arquitectónicas más importantes en los sistemas distribuidos modernos, pues ofrece la solución práctica al problema de la consistencia de datos cuando múltiples microservicios intervienen en un mismo proceso de negocio. En un sistema monolítico tradicional, todo el flujo puede encapsularse dentro de una transacción ACID única; sin embargo, en una arquitectura distribuida esto es imposible: cada microservicio conserva su propia base de datos, su propia lógica, su propio ciclo de despliegue y un estado independiente.

Intentar mantener consistencia estricta mediante transacciones distribuidas (two-phase commit, 2PC) introduce latencia significativa, cuellos de botella, bloqueos y, en general, compromete la escalabilidad. Por ello, las empresas modernas adoptan Sagas, que permiten realizar transacciones coordinadas sin necesidad de transacciones globales y con una estrategia clara para deshacer operaciones fallidas: las compensaciones.

EDA y Saga forman una pareja natural. La Saga describe *qué debe pasar y cómo revertirlo si ocurre un fallo*. La EDA determina *cómo se propagan las acciones mediante eventos*. En conjunto, permiten construir arquitecturas coherentes, tolerantes a fallos y con consistencia eventual, tal como requiere un sistema de clase empresarial como Amazon, Netflix o MercadoLibre.

1. Fundamentos del patrón Saga

Una Saga es, en esencia, una secuencia de transacciones locales distribuidas a través de varios microservicios. Cada uno realiza su parte como una transacción independiente y, al completarla, publica un evento que activa la siguiente fase. Si alguna etapa falla, se ejecutan operaciones de compensación que revierten parcial o totalmente la transacción previa.

Elementos clave:

1. **Transacción local:** Cada microservicio realiza operaciones ACID sobre su propia base de datos sin bloquear recursos globales.
2. **Eventos de progreso:** Se publican después de cada transacción exitosa para activar la siguiente.
3. **Consistencia eventual:** Los microservicios mantienen estados temporales y convergen a uno final una vez completada toda la secuencia.
4. **Acciones compensatorias:** Si ocurre un fallo, se ejecutan operaciones inversas. Ejemplo: si Inventario reservó 3 unidades pero el pago falló, se debe *desreservar*.
5. **Aislamiento distribuido:** Los microservicios nunca comparten bases de datos; cada uno es autónomo.
6. **Sin transacciones globales:** Una Saga sustituye el “todo o nada” ACID con un “todo o compensado”.

Este cambio no es trivial: implica pasar de un modelo determinista basado en transacciones a un modelo probabilístico basado en convergencia.

2. Tipos de implementación: Coreografiada vs Orquestada

Existen dos formas principales de implementar Sagas:

2.1. Saga Coreografiada

En este modelo no existe un controlador central. Los microservicios se coordinan solo mediante eventos: cada uno escucha eventos relevantes, ejecuta operaciones locales y publica eventos si tiene éxito o eventos compensatorios si falla.

Este estilo es coherente con el patrón de coreografía descrito en el punto 6.1.

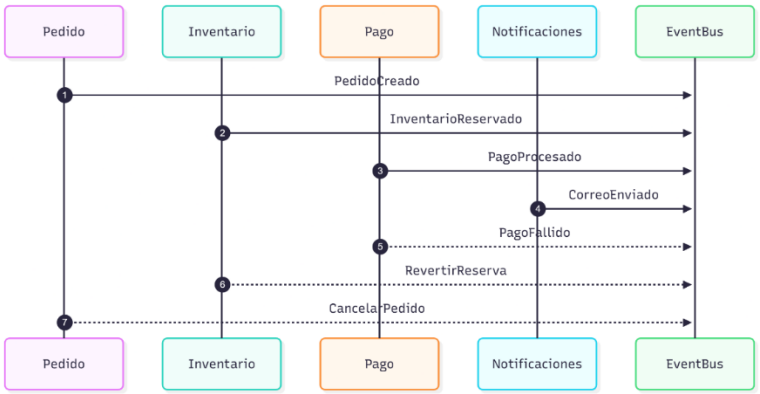
Ventajas:

- Desacoplamiento extremo.
- Fácil escalabilidad.
- Ideal para flujos altamente dinámicos y de gran volumen.

Desventajas:

- Complejidad emergente.
- Trazabilidad distribuida difícil sin OpenTelemetry.

Figura 4. Saga coreografiada en un flujo comercial distribuido.



Fuente: Elaboración propia (2025).

2.2. Saga Orquestada

Aquí existe un componente central llamado orquestador, responsable de:

- Ejecutar cada paso del flujo.
- Invocar a los microservicios.
- Evaluar resultados.
- Ejecutar compensaciones si ocurre un fallo.
- Registrar la secuencia del proceso.

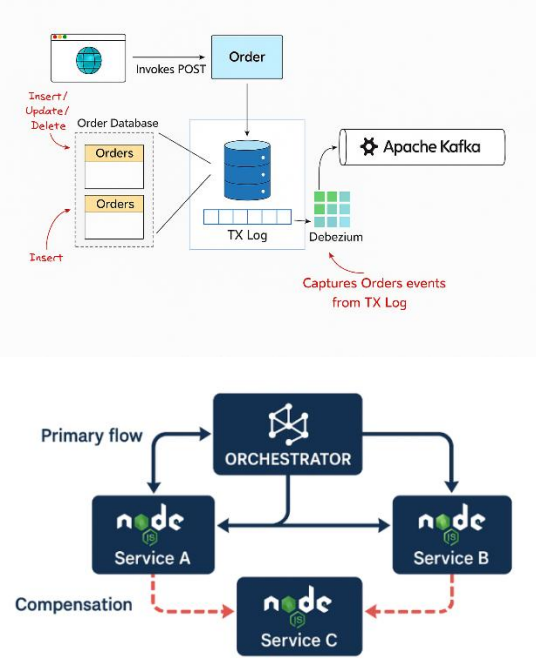
Ventajas:

- Visibilidad clara del proceso.
- Auditoría facilitada.
- Manejo explícito de errores.
- Ideal para flujos regulados (pagos, antifraude, contabilidad).

Desventajas:

- Riesgo de acoplamiento hacia el orquestador.
- Potencial punto único de fallo.
- Puede generar un “mini-monolito” si no se diseña bien.

Figura 5. Saga orquestada mediante un motor de flujo centralizado (alta resolución).



Fuente: Creación propia (2025).

3. Componentes estructurales de una Saga

Una implementación sólida del patrón Saga se basa en los siguientes elementos:

1. **Eventos de inicio**: suelen ser eventos de negocio (*PedidoCreado*).
2. **Transacciones locales**: cada servicio realiza un cambio atómico sobre su propio dominio.
3. **Eventos de transición**: *InventarioReservado*, *PagoProcesado*.
4. **Compensaciones**: *RestaurarInventario*, *CancelarPedido*, *ReembolsarPago*.
5. **Límites de decisión**: thresholds, timeouts, retries.
6. **Reglas de consistencia eventual**: el sistema converge a un estado estable.
7. **Reintentos idempotentes**: para no alterar el estado inesperadamente.

8. **Trazabilidad distribuida**: correlation-id por evento.

Tabla 2. Componentes del patrón Saga y su impacto arquitectónico.

Componente	**Función**	**Impacto en la arquitectura**
Transacción local	Modificación ACID	Autonomía por servicio
Evento de avance	Activa siguiente fase	Flujo distribuido
Evento compensatorio	Reversión parcial	Consistencia eventual
Orquestador	Control del proceso	Gobernanza central
Event Bus	Transporte asincrónico	Desacoplamiento

Fuente: Elaboración propia (2025).

4. Ejemplo aplicado: "Crear Pedido" en Amazon Distribuido

Este es uno de los flujos más característicos del comercio electrónico:

1. **PedidoService** crea un pedido y publica *PedidoCreado*.
2. **InventarioService** escucha el evento y reserva stock.
3. **PagoService** procesa el pago.
4. **LogísticaService** genera la orden de envío.
5. **NotificaciónService** envía la confirmación.

Compensación si ocurre un fallo:

- Si falla el pago → Inventario ejecuta *DesreservarStock*.
- Si falla inventario → Pedido se cancela sin intentar pago.
- Si falla logística → Se ejecuta un reembolso → Inventario se restaura.

Esta secuencia demuestra el principio fundamental de Saga:
Cada paso debe tener una operación inversa claramente definida.

5. El modelo matemático detrás de Saga

Desde una perspectiva formal, una Saga puede modelarse como:

- Una cadena de transiciones de estado $S_o \rightarrow S_1 \rightarrow \ldots \rightarrow S_n$
- Donde cada transición Ti es una operación local reversible.
- El sistema finaliza cuando alcanza un estado estable S_n o un estado estable compensado S'_k.

La garantía que ofrece Saga no es ACID, sino ACD (Atomicidad, Consistencia eventual, Durabilidad):

- **Atomicidad local**: cada transacción es indivisible.
- **Consistencia eventual**: el sistema converge.

- **Durabilidad**: cada paso deja un rastro en el Event Bus.

Este modelo matemático permite demostrar que incluso con fallos parciales, el sistema terminará eventual y determinísticamente en un estado estable (aunque no necesariamente el deseado inicialmente), lo cual es la esencia de la tolerancia a fallos distribuida.

6. Ventajas del Patrón Saga

1. Elimina transacciones distribuidas (2PC).
2. Escalabilidad extrema gracias al desacoplamiento.
3. Tolerancia a fallos mediante compensaciones.
4. Integración total con EDA.
5. Flexibilidad para evolucionar cada microservicio.
6. Permite flujos complejos sin bloquear recursos globales.

7. Desafíos del Patrón Saga

1. Complejidad conceptual elevada en flujos grandes.
2. Dificultad de depurar sin trazabilidad distribuida.
3. Dependencia de idempotencia en todos los consumidores.
4. Desorden potencial en la llegada de eventos.
5. Mayor tiempo hasta alcanzar la consistencia final.

Conclusión

El patrón Saga es fundamental para construir microservicios coherentes en sistemas altamente distribuidos. Su enfoque basado en eventos y compensaciones permite coordinar transacciones complejas sin incurrir en bloqueos, transacciones globales o acoplamientos inapropiados. En el proyecto Amazon Distribuido, Saga constituye el mecanismo que hace posible que pedidos, pagos, inventarios, logística y notificaciones cooperen y mantengan coherencia incluso ante fallos parciales.

6.3. Kafka y RabbitMQ para EDA (Event-Driven Architecture)

En una Arquitectura Orientada a Eventos (EDA), el componente técnico más fundamental es el Bus de Eventos, una infraestructura de mensajería que permite la comunicación asincrónica, desacoplada y basada en señales entre microservicios. Sin un Event Bus, la EDA sería imposible: los microservicios no podrían reaccionar a eventos, no existiría flujo distribuido y los sistemas caerían nuevamente en llamadas directas, acoplamientos rígidos y transacciones globales.

Dentro del ecosistema EDA contemporáneo, dos plataformas se han consolidado como pilares tecnológicos: RabbitMQ, un message broker tradicional orientado a colas y confiabilidad transaccional; y Apache Kafka, una plataforma de *event streaming* altamente escalable diseñada para manejar millones de eventos por segundo. Ambos cumplen roles distintos pero complementarios.

1. RabbitMQ: mensajería orientada a confiabilidad

RabbitMQ es un message broker basado en AMQP, cuyo objetivo principal es garantizar el enrutamiento, entrega, almacenamiento temporal y confirmación segura de mensajes. A diferencia de Kafka, que opera como un *commit log* distribuido, RabbitMQ está optimizado para coordinación transaccional y procesos que requieren fiabilidad en el corto plazo.

1.1. Naturaleza de RabbitMQ

RabbitMQ implementa un modelo de mensajería orientado a colas:

- Los productores envían mensajes a un exchange.
- El exchange decide a qué colas enviar cada mensaje.
- Las colas almacenan mensajes hasta que los consumidores puedan procesarlos.
- Los consumidores confirman la entrega (ACK).
- Si un consumidor falla, el mensaje vuelve a la cola o pasa a DLQ (Dead Letter Queue).

Este modelo es adecuado para la lógica transaccional presente en Sagas, como *reservar inventario*, *procesar pago*, *enviar correo*, etc.

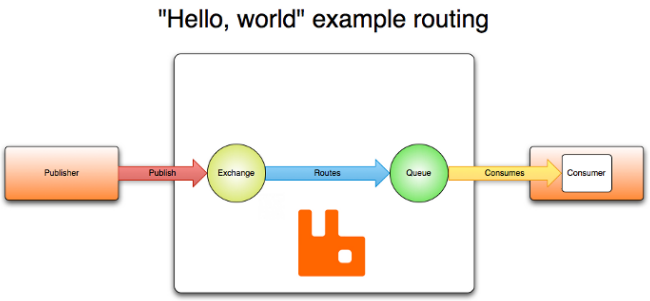
1.2. Garantías ofrecidas

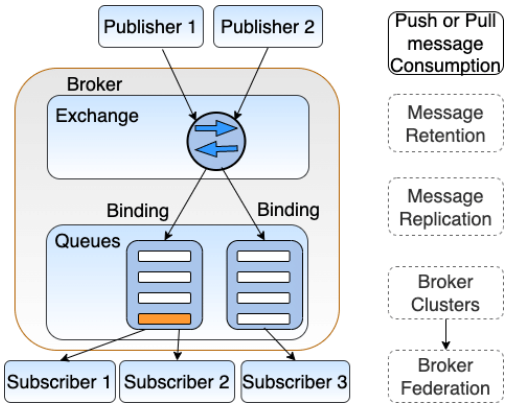
RabbitMQ provee:

- At-least-once delivery (Entrega al menos una vez)
- Orden relativo dentro de la cola
- Reintentos controlados
- Persistencia opcional
- Ruteo flexible (direct, fanout, topic, headers)

Estas características lo convierten en la mejor opción cuando la prioridad es la integridad semántica del flujo de negocio.

Figura 6. Arquitectura de RabbitMQ (exchange–queue–consumer).





Fuente: Elaboración propia (2025).

1.3. Implicaciones arquitectónicas

RabbitMQ es perfecto cuando:

- La lógica empresarial depende del orden de los mensajes.
- Se necesitan compensaciones de Saga precisas.
- Las transacciones deben ser confiables.
- El sistema trabaja con cargas moderadas-alta pero no en millones por segundo.

Es decir, RabbitMQ garantiza integridad operacional.

En Amazon Distribuido se utiliza para:

- PedidoCreado
- InventarioReservado
- PagoProcesado
- PagoFallido
- EnvíoProgramado

2. Apache Kafka: plataforma distribuida de event streaming

Apache Kafka no es un message broker tradicional. Es un log distribuido, diseñado para almacenar, replicar y transmitir eventos a un volumen masivo, manteniendo orden dentro de particiones y permitiendo a los consumidores “rebobinar” para reprocesar eventos.

2.1. Naturaleza de Kafka

Kafka funciona mediante:

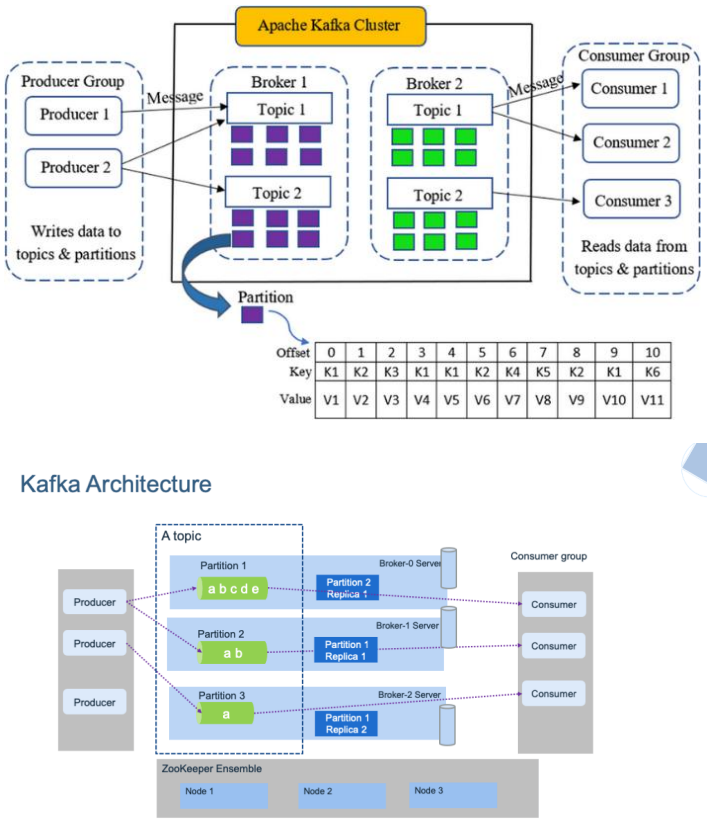
- Topics: canales de eventos.
- Particiones: segmentos ordenados y paralelos del topic.
- Offsets: punteros de lectura para cada consumidor.
- Brokers: nodos del clúster.
- Consumer Groups: escalamiento paralelo de consumidores.

Kafka no intenta garantizar entrega estricta como RabbitMQ. Kafka prioriza:

- Escalabilidad extrema
- Alto throughput
- Persistencia a largo plazo
- Procesamiento masivo
- Streaming en tiempo real

El consumidor decide qué offset leer, lo que permite análisis históricos, ML, recomendadores y auditorías.

Figura 7. Arquitectura de Kafka (producers → partitions → consumers).



Fuente: Creación propia (2025).

2.2. Garantías ofrecidas

Kafka asegura:

- Orden dentro de una partición.
- Replicación distribuida.
- Persistencia configurable (minutos a meses).
- Throughput extremadamente alto.
- Reprocesamiento ilimitado.

Por eso, Kafka es la base de:

- Sistemas de recomendación.
- Analítica de experiencia de usuario.
- Procesamiento de logs.
- Métricas de navegación.
- Observabilidad avanzada.

3. Comparación entre RabbitMQ y Kafka

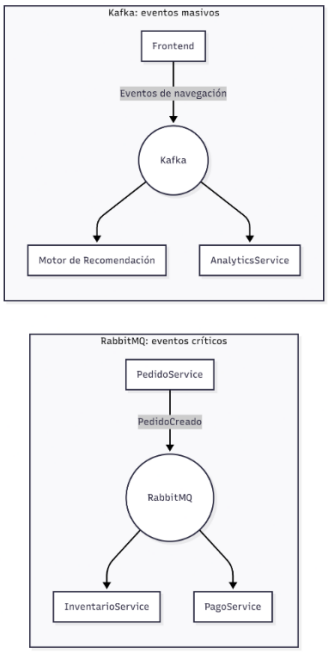
Tabla 3. Comparación teórica entre RabbitMQ y Kafka en EDA.

Criterio	**RabbitMQ**	**Kafka**
Modelo conceptual	Message Broker	Distributed Commit Log
Enfoque	Transaccional, confiable	Streaming masivo
Persistencia	Temporal	Durable
Orden	Por cola	Por partición
Prioridad	Integridad de negocio	Escalabilidad extremo
Reprocesamiento	Limitado	Nativo
Throughput	Alto	Muy alto
Uso ideal	Sagas, workflows	Analytics, ML, Telemetría

Fuente: Elaboración propia (2025).

4. Representación de RabbitMQ y Kafka

Figura 8. RabbitMQ y Kafka combinados en un flujo tipo Amazon.



Fuente: Elaboración propia (2025).

5. Uso combinado en un sistema empresarial

Una arquitectura madura casi siempre emplea ambos sistemas, porque resuelven problemas distintos. La dualidad RabbitMQ + Kafka permite:

- Integridad para eventos críticos (RabbitMQ).
- Escalabilidad para eventos masivos (Kafka).
- Resiliencia: si Kafka se satura, RabbitMQ sigue funcionando.
- Separación de dominios: negocio vs analítica.

La filosofía moderna es: "RabbitMQ para lo que el negocio no puede fallar.
Kafka para lo que el negocio necesita escalar."

6. Aplicación en el proyecto Amazon Distribuido

En el proyecto académico:

RabbitMQ se usa para:

- Procesos con Saga.
- Reservar inventario.
- Procesar pagos.
- Emitir notificaciones críticas.
- Coordinar pedidos.

Kafka se usa para:

- Historial de navegación.
- Recomendaciones basadas en comportamiento.
- Procesamiento de métricas.
- Telemetría del frontend.
- Análisis de patrones de consumo.

Ejemplo:

Un clic del usuario no debe afectar la consistencia del pedido → Kafka.
Un error en el pago no debe dejar datos inconsistentes → RabbitMQ.

7. Implicaciones arquitectónicas

7.1. Naturaleza del acoplamiento

RabbitMQ introduce un acoplamiento más determinista (aunque bajo), mientras Kafka fortalece un acoplamiento temporal casi nulo.

7.2. Resiliencia interna

RabbitMQ protege flujos críticos del negocio.
Kafka garantiza que el análisis no se detenga.

7.3. Impacto en la EDA

Una EDA completa necesita ambos:

- RabbitMQ ≈ brazo operativo (Sagas)
- Kafka ≈ cerebro analítico (streaming, ML)

7.4. Gobernanza de eventos

La coexistencia de ambos exige:

- AsyncAPI para contratos.
- Observabilidad distribuida.
- DLQs en RabbitMQ.
- Compaction y retención en Kafka.

8. Limitaciones y retos

RabbitMQ:

- No escala al nivel de Kafka.
- Requiere tuning más cuidadoso.
- No soporta replay avanzado.

Kafka:

- Demanda más hardware.
- No garantiza entrega estricta en transacciones críticas.
- Su complejidad operativa es mayor.

Conclusión

RabbitMQ y Kafka materializan dos filosofías complementarias dentro de la EDA. El primero ofrece confiabilidad transaccional, orden estricto y garantía operacional. El segundo proporciona escalabilidad extrema, persistencia duradera y capacidad para procesar flujos masivos de datos. Su uso combinado permite construir arquitecturas robustas, reactivas y preparadas para manejar millones de eventos, manteniendo integridad en procesos críticos sin sacrificar capacidad analítica ni velocidad.

En el proyecto Amazon Distribuido, ambos sistemas constituyen la base tecnológica que permite replicar escenarios reales de comercio electrónico, uniendo la precisión de las transacciones con el poder del análisis continuo.

6.4. Desacoplamiento mediante eventos

Uno de los principios más determinantes en las arquitecturas modernas es el desacoplamiento, entendido como la capacidad de permitir que módulos o microservicios evolucionen, escalen, fallen y se modifiquen sin impactar a los demás. La Arquitectura Orientada a Eventos (EDA) lleva este principio al extremo: en lugar de comunicar microservicios mediante invocaciones directas (REST, gRPC), los sistemas event-driven sustituyen la interacción explícita por la emisión y consumo de eventos.

El desacoplamiento mediante eventos representa un cambio epistemológico con implicaciones profundas: los microservicios dejan de "preguntarse" entre sí y empiezan a "informar" al ecosistema. De esta manera, el sistema global se vuelve más flexible, resiliente y escalable.

1. Concepto del desacoplamiento

El desacoplamiento consiste en reducir o eliminar dependencias directas entre los componentes de un sistema. En arquitecturas tradicionales:

- Un módulo A depende de un módulo B.
- Si B falla, A también falla.
- Si B cambia, A debe adaptarse.
- Si se escala B, A debe conocer la nueva ubicación.

En microservicios event-driven:

- A no depende de B.
- A publica eventos.
- B escucha eventos si los necesita.
- A no sabe cuántos consumidores existen ni quiénes son.

Este modelo elimina el acoplamiento:

- lógico,
- temporal,
- semántico,
- espacial.

Por ello, EDA es el paradigma más avanzado para sistemas masivos.

2. Tipos de desacoplamiento habilitados por EDA

El desacoplamiento mediante eventos se manifiesta en cuatro niveles:

2.1. Desacoplamiento espacial

El productor no necesita conocer la dirección, endpoint, protocolo o localización del consumidor.

Ejemplo típico en Amazon Distribuido:

- PedidoService publica PedidoCreado.
- No importa si Inventario está en otra región, VPC o clúster.
- Solo debe publicarse el evento.

Este principio permite:

- Multi-regionalidad nativa.
- Replicación geográfica.
- Conectividad sin dependencias duras.

2.2. Desacoplamiento temporal

El consumidor puede estar fuera de línea, saturado o en proceso de despliegue y aún así no se pierde el evento. El Event Bus lo almacena temporalmente.

Esto garantiza:

- Tolerancia a fallos.
- Asincronía completa.
- Elasticidad operacional.

2.3. Desacoplamiento semántico

El significado de un evento puede ser distinto para cada consumidor.

Ejemplo:

- "PedidoCreado" significa:
    - Para Inventario → reservar stock
    - Para Analytics → incrementar dashboard
    - Para Marketing → activar embudo de bienvenida
    - Para ML → alimentar modelos de predicción

El productor no tiene por qué conocer estos significados.

2.4. Desacoplamiento evolutivo

Los microservicios pueden:

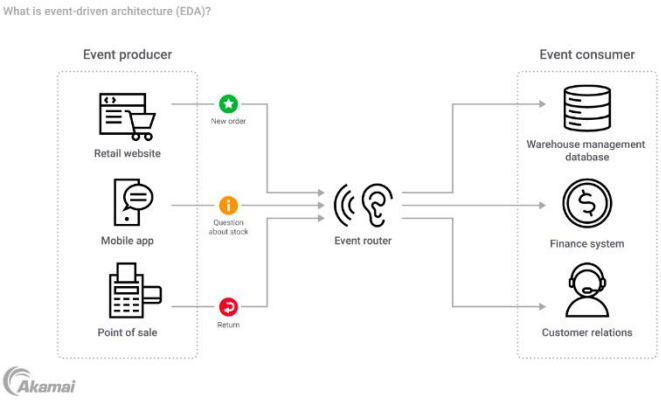
- Cambiar de tecnología
- Reescribirse
- Escalar
- Dividirse
- Fusionarse
- Desplegarse independientemente

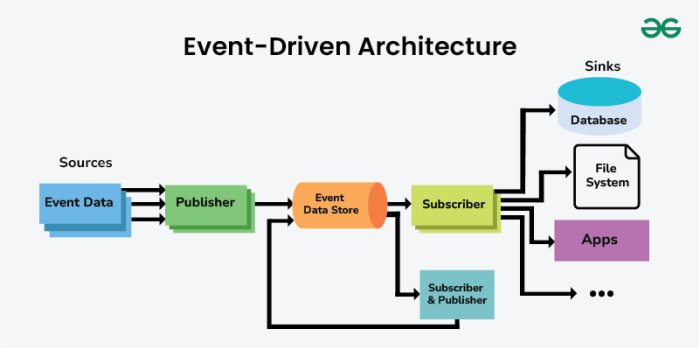
Mientras mantengan los mismos eventos contractuales, el sistema no se rompe.

Este es el fundamento que permite la evolución acelerada de plataformas como Amazon y Netflix.

✔ 3. Representación visual del desacoplamiento

Figura 9 .Desacoplamiento estructural mediante un Event Bus.





Fuente: Creación propia (2025).

En esta figura se muestra que los servicios nunca se llaman entre sí directamente. Todo fluye a través del Event Bus, lo que elimina dependencias rígidas y permite que el sistema continúe incluso con servicios intermitentes.

4. Ventajas arquitectónicas del desacoplamiento

4.1. Resiliencia y tolerancia a fallos

En sistemas acoplados, un fallo en un servicio afecta al resto.
En EDA:

- El productor no depende del consumidor.
- Un consumidor lento no detiene el sistema.
- Un broker robusto (Kafka/RabbitMQ) absorbe picos de carga.

4.2. Escalabilidad horizontal ilimitada

Los consumidores pueden multiplicarse según carga:

- Inventario puede tener 4 instancias

- Pago puede tener 20
- Notificaciones 50

Cada uno procesa eventos al ritmo que puede.

4.3. Flexibilidad y extensibilidad

Nueva funcionalidad = nuevo consumidor.
No se toca el productor.

Ejemplo real:

Se desea implementar recomendaciones:

→ Se crea un microservicio *RecomendadorService* que escucha *PedidoCreado*.
→ No se modifica PedidoService.

4.4. Simplicidad para los productores

Los productores solo informan:
"Esto pasó".
No deben saber qué ocurrirá después.

5. Tabla comparativa: acoplamiento vs desacoplamiento

Tabla 4. Contraste entre sistemas acoplados y desacoplados mediante eventos.

Criterio	**Sistema acoplado**	**Sistema desacoplado (EDA)**
Dependencias	Fuertes	Mínimas
Fallos	Cascada	Aislados
Evolución	Lenta	Rápida
Escalabilidad	Limitada	Alta
Conocimiento del sistema	Completo y rígido	Parcial y flexible
Coordinación	Llamadas directas	Eventos
Resiliencia	Baja	Alta

Fuente: Elaboración propia (2025).

6. Desacoplamiento aplicado al proyecto Amazon Distribuido

En este curso, el desacoplamiento se materializa en varios flujos:

6.1. Flujo de pedido

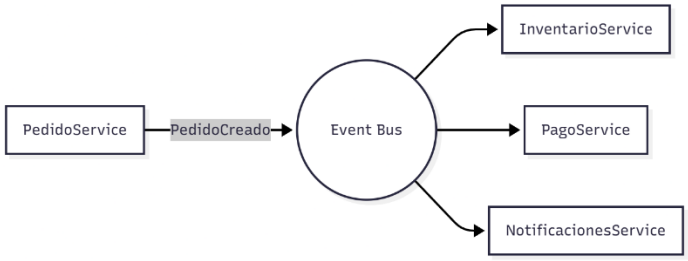
1. PedidoService publica **PedidoCreado**
2. InventarioService actúa: **ReservaStock**
3. PagoService responde: **PagoProcesado**
4. LogísticaService programa envío

5. NotificacionesService informa al usuario

Resultado clave:

- Ningún servicio conoce al otro.
- Solo responden a eventos.
- Permite que Inventario escale 5× sin modificar el resto.

Figura 10. Flujo desacoplado de un pedido en Amazon Distribuido.



Fuente: Elaboración propia (2025).

7. Implicaciones

7.1. Modelo sistema → ecosistema

En sistemas acoplados, los microservicios son nodos que forman un grafo dirigido con alta densidad. En EDA, el sistema se convierte en un ecosistema, donde:

- Las relaciones son difusas.
- Los comportamientos son emergentes.
- La dinámica evolutiva es flexible.
- Los límites de dominio se vuelven más claros.

7.2. Reducción del acoplamiento cognitivo

Menos dependencias → menor carga cognitiva:

- Un desarrollador no necesita entender 10 APIs.
- Solo eventos relevantes a su microservicio.

Esto acelera el desarrollo.

7.3. Implicaciones matemáticas

La EDA transforma el sistema en:

- Un grafo bipartito dirigido (productores → Event Bus → consumidores).
- Con propiedades de resiliencia similares a redes biológicas y redes eléctricas.

Resiliencia ≈ redundancia + desacoplamiento.

8. Limitaciones del desacoplamiento

No todas son ventajas. El desacoplamiento trae retos:

- Mayor complejidad para depurar flujos distribuidos.
- Dependencia absoluta de la observabilidad (OpenTelemetry).
- Necesidad de idempotencia estricta.
- Riesgo de eventos fantasma o procesados doble.
- Posible inconsistencia temporal (se acepta consistencia eventual).

Conclusión

El desacoplamiento mediante eventos es un principio esencial en sistemas distribuidos modernos. Al eliminar dependencias directas, se habilita un entorno donde los microservicios pueden fallar, escalar y evolucionar sin comprometer la coherencia global del sistema. Esto permite que arquitecturas como Amazon, Netflix o MercadoLibre funcionen a una escala que sería imposible con acoplamientos tradicionales.

En el contexto del proyecto Amazon Distribuido, el desacoplamiento permite implementar flujos complejos de pedidos, pagos, inventario, logística y notificaciones sin exigir que los microservicios se conozcan entre sí, garantizando eficiencia, resiliencia y escalabilidad.

6.5. Monitoreo y trazabilidad en arquitecturas orientadas a eventos

La Arquitectura Orientada a Eventos (EDA) presenta beneficios extraordinarios para la escalabilidad y resiliencia de los sistemas distribuidos. Sin embargo, esta misma naturaleza descentralizada introduce un desafío que no existe en sistemas monolíticos ni en arquitecturas con control centralizado: ¿cómo comprender, seguir y diagnosticar el comportamiento del sistema cuando las interacciones ocurren de forma asincrónica, distribuida y emergente?

En una EDA, los microservicios no se llaman directamente. No hay una cadena REST clara, no hay un orquestador que dirija todo el flujo, y no existen transacciones centralizadas. En su lugar, hay eventos que van y vienen, múltiples consumidores con estados intermedios, reintentos, colas, particiones, offsets, DLQs y varias capas de cachés. Esto convierte al diagnóstico en una tarea compleja que exige herramientas, métricas, estándares y técnicas avanzadas de observabilidad.

La trazabilidad distribuida es un componente vital del éxito de toda arquitectura moderna. Sin ella, un sistema puede funcionar, pero no es capaz de evolucionar adecuadamente. La observabilidad no solo permite detectar fallos: permite entender cómo se comporta el sistema, cómo escalarlo y cómo garantizar consistencia eventual.

1. Concepto de observabilidad en EDA

La observabilidad se refiere a la capacidad de un sistema distribuido para exponer suficiente información interna que permita inferir su estado, comportamiento, causas de errores y performance. En la EDA, observar significa poder reconstruir flujos que no existen como llamadas directas, sino como conjuntos de eventos diseminados en el tiempo y en diferentes microservicios.

1.1. Diferencia entre monitoreo y observabilidad

- **Monitoreo:** recolecta métricas predefinidas (CPU, RAM, latencia).

- **Observabilidad:** permite responder preguntas que no fueron anticipadas.
  Ejemplo: ¿por qué los pedidos de Guayaquil tardan más en la fase InventarioReservado?

1.2. El desafío de la asincronía

En REST, un flujo típico es:

```
A → B → C → D
```

En EDA, un flujo puede ser:

```
Evento X → Servicio A → Evento Y → Servicio C → Evento Z → Servicio B → Evento W → Servicio D
```

Es imposible inferir este flujo sin mecanismos avanzados.

2. Trazabilidad distribuida

La trazabilidad distribuida consiste en seguir el "rastro" de un evento a través de múltiples servicios. Esto se logra mediante:

- Correlation IDs
- Trace IDs y Span IDs
- Context propagation
- Logs estructurados
- Sistemas de visualización de trazas

Correlation ID

Cada evento necesita un identificador único que permita rastrear su propagación. Este ID debe viajar:

- En el mensaje RabbitMQ
- En el mensaje Kafka
- En los logs
- En los headers internos
- En cualquier respuesta que genere

El correlation ID es la columna vertebral de la trazabilidad.

3. Componentes esenciales del monitoreo en EDA

3.1. Logs estructurados

Los logs tradicionales en texto plano son inadecuados. Los sistemas distribuidos requieren logs con estructura:

- JSON
- Campos: timestamp, trace_id, span_id, correlation_id, service, event_name, duration, payload_size

- Enviados a un sistema central (ELK, Loki, CloudWatch)

Los logs deben reflejar:

- Recepción de eventos
- Envío de eventos
- Errores al procesar
- Retrasos
- Reintentos

3.2. Métricas

Las métricas son indicadores cuantitativos:

- Tiempo promedio de procesamiento por evento
- Cantidad de eventos procesados por minuto
- Número de mensajes en cola
- Latencia entre productor y consumidor
- Fallos por tipo de evento
- Retrasos de particiones Kafka
- Tamaño de DLQs

Estas métricas permiten detectar cuellos de botella y realizar escalamiento proactivo.

3.3. Trazas distribuidas

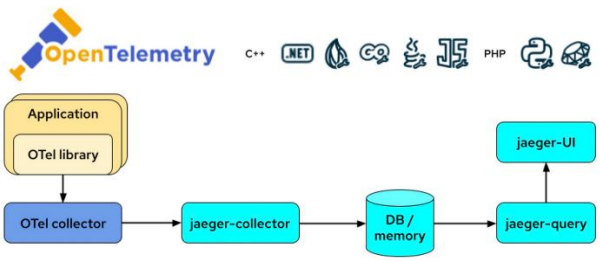
Las trazas (traces) permiten reconstruir la secuencia de pasos que siguió un evento.

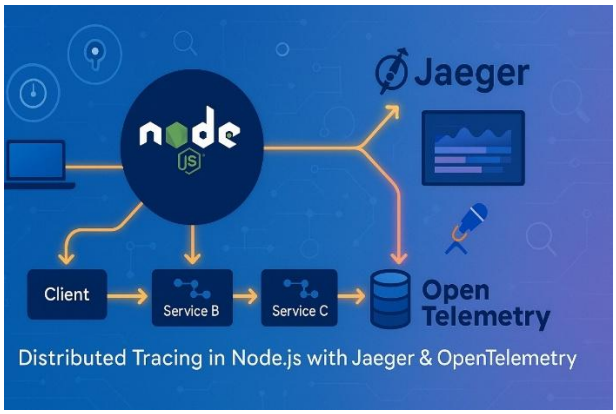
Cada paso de la traza se denomina span:

- Span de PedidoService
- Span de Inventario
- Span de Pago
- Span de Notificaciones

Cuando se visualizan en un grafo temporal, permiten observar la propagación del evento.

Figura 11. Representación de trazabilidad distribuida en EDA.





Fuente: Creación propia (2025).

4. Herramientas fundamentales

4.1. OpenTelemetry

Es el estándar moderno para:

- Recolección de logs
- Propagación de contexto
- Generation de trace_id y span_id
- Métricas
- Exportación a sistemas de monitoreo

Aborda uno de los problemas históricos de los microservicios: la falta de uniformidad.

4.2. Jaeger

Plataforma para visualizar trazas distribuidas:

- Muestra diagramas de Gantt de los spans
- Permite filtrar por trace_id
- Revela servicios lentos
- Detecta patrones anómalos

4.3. Prometheus + Grafana

- Prometheus recolecta métricas.
- Grafana las visualiza.

Se crean dashboards como:

- Latencia total del flujo “CrearPedido”
- Retraso de colas RabbitMQ
- Lag de particiones Kafka
- Fallos por microservicio

- Throughput del sistema

4.4. ELK (Elasticsearch, Logstash, Kibana)

Ideal para análisis profundo de logs estructurados.

5. Monitoreo y trazabilidad aplicados al proyecto Amazon Distribuido

En el proyecto del curso:

5.1. Cada evento RabbitMQ incluye:

- correlation_id
- nombre del evento
- timestamp de emisión
- timestamp de recepción
- payload minimizado

5.2. Cada evento Kafka incluye:

- offset
- partición
- correlation_id
- metadata adicional

5.3. Cada microservicio mantiene logs estructurados:

Ejemplo:

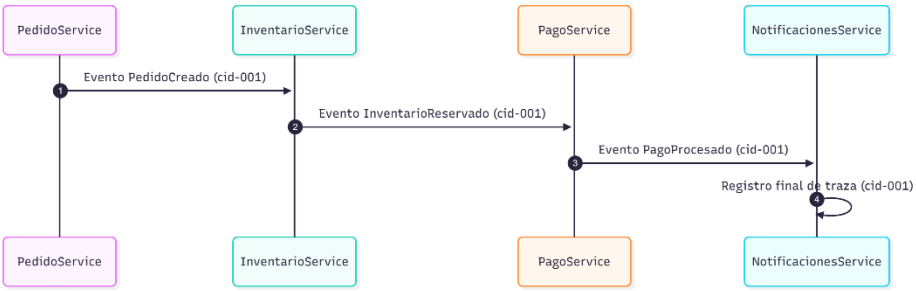
```
{
  "timestamp": "2025-11-28T16:44:01Z",
  "service": "InventarioService",
  "event_received": "PedidoCreado",
  "correlation_id": "cid-8821a",
  "status": "processed",
  "duration_ms": 42
}
```

5.4. Trazas distribuidas

La herramienta Jaeger permite visualizar:

- Flujo completo de un pedido desde PedidoService hasta NotificaciónService
- Identificación del microservicio más lento
- Retrasos en Inventario o Pago
- Retrasos en el Event Bus

Figura 12. Flujo trazado de un pedido en Amazon Distribuido.



Fuente: Elaboración propia (2025).

6. Retos profundos del monitoreo en EDA

6.1. Fragmentación de información

El estado está distribuido, no centralizado.

6.2. Consistencia eventual

El sistema puede estar en estados intermedios no triviales.

6.3. Latencias variables

Los flujos asincrónicos generan tiempos no deterministas.

6.4. Volumen de datos

EDA produce millones de eventos → millones de logs → terabytes de información.

6.5. Correlación compleja

Una traza puede involucrar:

- RabbitMQ
- Kafka
- 6 microservicios
- 3 reintentos
- 1 compensación

Reconstruir la historia completa sin herramientas es imposible.

7. Estrategias recomendadas para observabilidad en EDA

1. Correlation ID obligatorio en todos los eventos.
2. Logs JSON estructurados en todos los microservicios.
3. Propagación de contexto mediante OpenTelemetry.
4. Dashboards especializados por flujo (CrearPedido, ReembolsarPedido, etc.).
5. Alertas proactivas:

    - DLQ > 10 mensajes.
    - Latencia Kafka > 2s.
    - Caída de throughput.
6. Trazabilidad cross-broker (RabbitMQ + Kafka).
7. Monitoreo del Event Bus como componente crítico.

Conclusión

El monitoreo y la trazabilidad son elementos indispensables para operar y evolucionar arquitecturas orientadas a eventos. La asincronía, la distribución y la consistencia eventual introducen complejidad que solo puede manejarse mediante observabilidad avanzada. Un sistema event-driven sin trazabilidad está condenado al caos operacional: los errores son invisibles, los cuellos de botella permanecen ocultos y los procesos críticos se vuelven imposibles de auditar.

Al implementar mecanismos robustos de correlación, logs estructurados, métricas, trazas distribuidas y visualización, se logra un ecosistema controlado, estable y predecible. En Amazon Distribuido, estas prácticas permiten reconstruir flujos enteros de eventos, identificar problemas con precisión y garantizar una operación confiable incluso bajo alta concurrencia.

Referencias citadas en la Clase 6

- Apache Software Foundation. (2024). *Apache Kafka Documentation*. https://kafka.apache.org
- RabbitMQ. (2024). *RabbitMQ Tutorials and Reliability Guide*. https://www.rabbitmq.com
- New Relic. (2024). *Understanding Distributed Tracing in Microservices*.
- OpenTelemetry. (2024). *OpenTelemetry Specification*. https://opentelemetry.io
- Google Cloud. (2024). *Event-Driven Architecture Patterns and Best Practices*.
- AWS. (2024). *Building Event-Driven Architectures on AWS*.
- Netflix Tech Blog. (2024). *Adaptive Distributed Systems Using Event-Driven Principles*.
- Confluent. (2024). *Designing Event Streaming Architectures with Kafka*.
- Microsoft Azure. (2024). *Enterprise Integration Patterns and Event-Driven Systems*.
- Fowler, M. (2023). *Patterns of Distributed Systems: Events, Sagas and Workflow Models*.

Definición de términos

- **EDA (Event-Driven Architecture):** Estilo arquitectónico basado en la producción, detección y reacción a eventos distribuidos.
- **Evento:** Hecho significativo del dominio que indica un cambio de estado (*PedidoCreado*, *PagoProcesado*).
- **Event Bus:** Canal de transporte de mensajes que soporta la difusión y consumo de eventos (Kafka, RabbitMQ).
- **Saga:** Patrón que permite coordinar transacciones distribuidas mediante pasos locales y acciones compensatorias.
- **Coreografía:** Coordinación descentralizada donde los microservicios reaccionan a eventos sin un controlador central.

- **Orquestación:** Coordinación centralizada mediante un orquestador que controla el flujo de ejecución.
- **DLQ (Dead Letter Queue):** Cola donde se almacenan eventos que fallaron repetidamente en su procesamiento.
- **Offset:** Posición exacta de lectura de un evento dentro de una partición Kafka.
- **Span / Trace:** Unidades de seguimiento en trazabilidad distribuida para reconstruir flujos de ejecución.
- **Correlation ID:** Identificador único que permite rastrear un evento a través de múltiples microservicios.
- **Idempotencia:** Propiedad que asegura que procesar el mismo evento varias veces no cambia el estado final.
- **Backpressure:** Mecanismo que controla la presión entre productores y consumidores para evitar saturación.

Recursos de profundización

Recurso: *Profundizacion1_Clase6_EDA.docx*
Contenido sugerido:

- Ejemplos detallados de Sagas (coreografiadas y orquestadas).
- Comparación técnica entre Kafka, RabbitMQ, Pulsar y Kinesis.
- Práctica avanzada de trazabilidad distribuida con OpenTelemetry + Jaeger.
- Diseño de DLQ, estrategias de reintento, y patrones de resiliencia.
- Caso de estudio: Arquitectura event-driven de Amazon.

ACTIVIDAD DE EVALUACIÓN

INSTRUCCIONES

Responde las siguientes preguntas de opción múltiple relacionadas con los temas abordados en la Clase 6. Selecciona la alternativa correcta en cada caso. Finalmente, revisa la retroalimentación para reforzar tu comprensión.

Título: Arquitectura Orientada a Eventos (EDA) y microservicios

Dificultad: Media
RDAs evaluados:

- RDA 1
- ✓ RDA 2
- RDA 3

PREGUNTA 1

¿Cuál es el objetivo principal de la Arquitectura Orientada a Eventos (EDA) en sistemas distribuidos?

CORRECTA:
Permitir la comunicación desacoplada entre microservicios mediante la emisión y consumo de eventos.

Incorrecta:
Garantizar consistencia inmediata mediante transacciones ACID centralizadas.

Incorrecta:
Eliminar la necesidad de mecanismos de monitoreo.

Incorrecta:
Reemplazar completamente REST y gRPC en todas las arquitecturas.

Retroalimentación:
La EDA permite que los microservicios cooperen sin depender de llamadas directas, promoviendo escalabilidad, resiliencia y asincronía.

PREGUNTA 2

¿Qué función principal cumple el patrón Saga en una arquitectura distribuida?

CORRECTA:
Coordinar transacciones distribuidas mediante pasos locales y acciones compensatorias.

Incorrecta:
Centralizar todos los flujos de negocio en un solo orquestador.

Incorrecta:
Garantizar procesamiento de eventos en orden global absoluto.

Incorrecta:
Reemplazar DLQs y mecanismos de reintento.

Retroalimentación:
Saga evita el uso de transacciones globales y permite coherencia eventual a través de compensaciones.

PREGUNTA 3

¿Qué característica distingue a Kafka frente a RabbitMQ en sistemas event-driven?

CORRECTA:
Permite almacenamiento persistente y reprocesamiento masivo mediante particiones y offsets.

Incorrecta:
Es el más adecuado para flujos transaccionales críticos.

Incorrecta:
Elimina la necesidad de correlation IDs.

Incorrecta:
Opera exclusivamente bajo el protocolo AMQP.

Retroalimentación:
Kafka está diseñado para streaming masivo y persistente, ideal para analítica y ML a gran escala.

PREGUNTA 4

¿Cuál es el propósito del correlation ID en trazabilidad distribuida?

CORRECTA:
Permitir rastrear un evento a través de múltiples microservicios y reconstruir su flujo completo.

Incorrecta:
Garantizar la seguridad criptográfica de los mensajes.

Incorrecta:
Reducir el tamaño del payload.

Incorrecta:
Evitar la creación de logs en los microservicios.

Retroalimentación:
El correlation ID es esencial para reconstruir flujos asincrónicos y visualizar trazas completas con herramientas como Jaeger.

Enlaces externos recomendados

Título del enlace relacionado: Event-Driven Architectures on AWS
Descripción: Guía oficial sobre patrones EDA, Sagas, diseño de Event Bus y mensajería distribuida en sistemas empresariales.
Enlace: https://aws.amazon.com/event-driven-architecture

Título del enlace relacionado: Apache Kafka – Official Documentation
Descripción: Documentación oficial para la construcción de arquitecturas de streaming, manejo de particiones, offsets y análisis en tiempo real.
Enlace: https://kafka.apache.org/documentation

Título del enlace relacionado: RabbitMQ – Reliability and Messaging Patterns
Descripción: Guía oficial para implementar consumer acknowledgements, DLQ, reintentos y patrones de mensajería empresarial.
Enlace: https://www.rabbitmq.com/docs

Título del enlace relacionado: OpenTelemetry – Distributed Tracing
Descripción: Estándar abierto para instrumentación de microservicios, propagación de contexto y generación de spans.
Enlace: https://opentelemetry.io

Título del enlace relacionado: Netflix Tech Blog – Observability at Scale
Descripción: Análisis de cómo Netflix monitorea millones de eventos, microservicios y flujos event-driven en tiempo real.
Enlace: https://netflixtechblog.com